Indian Academy of Sciences
C.V. Raman Avenue, P.B. No. 8005, Bangalore 560 080

Exploring Dynamic Graph Neural Networks for Seizure Prediction: An Investigative Study

FASTSF 2025
Project Report

Indian Institute of Space Science and Technology, Thiruvananthapuram

Name: Poorvai Chandrasen
Application No: FENGS156
Indian Institute of Science Education and Research, Bhopal

Instructor: Dr. Deepak Mishra
Indian Institute of Space Science and Technology, Thiruvananthapuram
Date: 18.08.2025

Declaration:

I hereby declare that the project report titled *Exploring Dynamic Graph Neural Networks for Seizure Prediction: An Investigative Study*, submitted by me, **Poorvai Chandrasen**, a bonafide student of the Indian Institute of Science Education and Research, Bhopal, is an original work carried out by me under the guidance of Dr, Deepak Mishra, Professor at the Indian Institute of Space Science and Technology, Thiruvananthapuram. This report has not been submitted previously to any other university or institution for the award of any degree, diploma, or fellowship.

Poorvai Chandrasen

BS Engineering, IISER Bhopal

Date: August 18, 2025 Place: Bhopal

Acknowledgments:

I would like to extend my heartiest gratitude to Dr. Deepak Mishra, Professor, Dept. of Avionics, IIST, Thiruvananthapuram, for his invaluable guidance, and to Mr. Ramzan Basheer, Ph.D. scholar at the Dept. of Avionics, IIST, Thiruvananthapuram, for the insightful discussions from which I learned to appreciate studying.
I am grateful to the Indian Academy of Sciences for providing the opportunity to undertake the Summer Research Fellowship 2025.
Finally, I thank my family for their constant encouragement and unwavering support, as well as my colleagues in the VR Lab who were an indispensable source of motivation, joy, and support. Without them this work would not have been possible.

Poorvai Chandrasen
IISER, Bhopal

Contents

1 AIM **5**

2 INTRODUCTION **5**

3 BACKGROUND **5**

4 Theoretical Framework **5**
4.1 Graph Theory Fundamentals . . . 5
4.2 Graph Kernels . . . 7
4.3 Graph Neural Networks . . . 8

5 Literature Review **9**
5.1 Adjacency matrix Construction . . . 9
5.1.1 Pearson Correlation . . . 10
5.1.2 Spearman Rank Correlation . . . 10
5.1.3 Distribution Parameter difference . . . 11
5.1.4 Mutual Information . . . 11
5.2 Node Feature encoding . . . 11
5.2.1 GCNConv . . . 11
5.2.2 GINConv . . . 12
5.2.3 GraphCapsule . . . 13
5.3 Unified Architecture . . . 13
5.3.1 GNN-RNN Combinations . . . 14
5.3.2 Structural Specific Models . . . 15
5.3.3 Temporal Specific Models . . . 15

6 Methodology **15**
6.0.1 Motivation for specific model choices . . . 15
6.0.2 Data Preprocessing . . . 16
6.0.3 Graph Construction . . . 17
6.0.4 Model Architecture . . . 17
6.0.5 Training Setup . . . 17

7 Results **18**

8 Conclusion **25**

9 Future Scope **26**

1 AIM

This project examines the effectiveness of Dynamic Graph Neural Networks (GNNs) for seizure prediction using a publicly available EEG dataset. We evaluate different node feature encoding strategies, graph construction methods, and dynamic modeling approaches.

2 INTRODUCTION

Seizures are sudden, uncontrolled electrical disturbances in the brain causing changes in behavior, movements, or consciousness, often linked to epilepsy [1]. Affecting about 50 million worldwide [6], seizures can impair health by causing physical injuries, cognitive decline, and emotional distress, while reducing quality of life through social stigma, employment challenges, and restricted independence. Thus, detection and prediction become a priority for enabling timely intervention.

Electroencephalogram (EEG) datasets are used to obtain data for electrical activity in the brain. Methods to classify seizure using EEG evolved from Convolutional Neural Networks (CNNs) to Graph Neural Networks (GNNs), later combined with time series models to capture both spatial and temporal data. However, combining dynamic connectivity in graphs with temporal data to efficiently capture the information in the dataset remained as a challenge.

3 BACKGROUND

Electric signals in the brain may be classified into the following states [2]:
<u>Pre-ictal</u> signals: occurring minutes to hours before a seizure, indicating a transitional brain state potentially predictive of an impending seizure.
<u>Ictal</u> signals: during a seizure, characterized by abnormal, synchronized electrical activity in the brain.
<u>Post-ictal</u> signals: following a seizure, often showing slowed activity or recovery as the brain returns to baseline.
<u>Interictal</u> signals: recorded between seizures, reflecting baseline brain activity without ictal (seizure) events.

Prediction of a seizure is often paraphrased as the classification of the input signal as a pre-ictal or an interictal signal. Correct identification of pre-ictal signal can alert the patient of an impending seizure (cluster), and preventative measures can be taken.

4 Theoretical Framework

4.1 Graph Theory Fundamentals

A graph is a mathematical structure used to model relationships between entities. It consists of a set of individual elements, referred to as vertices (or nodes), and a set of edges that define pairwise relationships between these vertices according to a specified criterion. Graph structures are inherently non-linear and non-Euclidean, meaning they do not conform to the constraints of regular grids or coordinate systems.
Any Graph is defined as $G = (V, E, X)$: Where
V: set of vertices

E: set of edges connecting any pair of vertices following a relation defined on the graph
X: features of each vertex/node

Graphs can be categorized into several types:

- Simple graph: contains at most one edge between any pair of vertices.
- Multigraph: permits multiple edges between the same pair of vertices.
- Directed graph (digraph): edges have an orientation, allowing information to flow only from v_i to v_j
- Undirected graph: edges have no direction, enabling bidirectional information flow between vertices.
- Weighted graph: each edge is assigned a numerical value (weight), representing properties such as cost, distance, or importance.

The adjacency matrix is a common representation of graph connectivity. A graph with N vertices is denoted by $A_{N\times N}$ square matrix, where each entry $a_{i,j}$ indicates the presence (and, if applicable, the weight) of an edge between vertices v_i and v_j.
Given a graph $G = (V, E)$ with $|V| = N$, the adjacency matrix $A \in \mathbb{R}^{N\times N}$ is defined as:

$$a_{ij} = \begin{cases} 1, & \text{if } (v_i, v_j) \in E, \\ 0, & \text{otherwise.} \end{cases}$$

For a weighted graph, each $a_{ij} = w_{ij} \in \mathbb{R}$ denotes the weight of the edge between vertices v_i and v_j.
In an undirected graph, the adjacency matrix A is symmetric, that is, $a_{ij} = a_{ji}$ for all i, j.

Why Represent EEG Data as Graphs?

Neurons in the brain generate electrical impulses to transmit information. Electrodes placed on the scalp record the resulting electrical potentials arising from neural activity. Electroencephalography (EEG) is a technique for capturing these signals, enabling the identification of abnormal neural patterns.

EEG recordings can be organized into a matrix $\mathbf{X} \in \mathbb{R}^{n\times T}$, where each of the n rows corresponds to an electrode (channel) and each of the T columns represents voltage samples over time.
Convolutional Neural Networks (CNNs) have traditionally been employed for EEG classification. Given an input matrix X, convolution layers may be applied in one dimension (across time) or in two dimensions (across both time and channels). In another approach, the CNN processes the entire EEG signal by first converting it into a spectrogram, paired with its label (pre-ictal or inter-ictal). The network then extracts features from the spectrogram in the same way it does from an image and classifies the test signal based on the learned feature representations
However, EEG data are inherently non-Euclidean. A temporal matrix-based representation fails to explicitly encode the spatial relationships between electrodes[9].

Graph-based representations address this limitation by modeling EEG channels as nodes in a graph, with edges encoding relationships such as physical distances between electrodes, functional connectivity (e.g., correlation, coherence), or frequency-domain similarity. Consequently, the objective of this project is to classify EEG graphs as either pre-ictal or inter-ictal by learning to identify structural and feature-based differences between the two types of graphs. This problem is closely related to the concept of graph isomorphism.
Two graphs G and H are isomorphic if there exists a bijection $\phi : V(G) \to V(H)$: such that any two vertices u and v are adjacent in G if and only if $\phi(u)$ and $\phi(v)$ are adjacent in H.

While exact graph isomorphism testing can be computationally challenging, many approaches in graph kernels and message passing neural networks aim to learn graph representations that implicitly capture such structural distinctions.

4.2 Graph Kernels

Graph kernels are mathematical functions that quantify the similarity between two graphs by computing an inner product in an implicit feature space. They enable the representation of non-linear graph structures into a feature space.
For eg: WL kernel, Random Walk kernel A key property of many graph kernels is that they operate on substructures of graphs that can be computed in polynomial time, thereby ensuring computational tractability [8]. These kernels are non-parametric, meaning they do not learn parameters from data, but rather define a fixed similarity function to process Graph Data.

A widely used example of a graph kernel is the Weisfeiler–Lehman (WL) kernel. It is based on node-label refinement and iterative neighborhood aggregation to capture progressively richer structural information. The WL kernel operates by repeatedly updating node labels so that they encode the local topology of the graph, and then comparing the distributions of these labels across graphs to measure similarity.
Formally, the WL kernel transforms the input graphs through h iterations of label refinement, where at each iteration:

1. Initialization – Assign each node an initial label (uniform for unlabeled graphs).
2. Label Refinement: Over each iteration $i = 1, \ldots, k$:
   - Update each node's label by hashing its current label together with the multiset of its immediate neighbors' labels.
   - This step encodes broader structural context iteratively.
3. Feature Extraction – After each iteration, count the occurrences of each distinct label across the graph. These label frequency vectors serve as features.
4. Termination Conditions – Two stopping criteria apply:
   - Reaching a predefined maximum number of iterations k (a hyperparameter reflecting desired receptive field depth).
   - Label convergence, where node labels remain unchanged between successive iterations.
5. Kernel Computation – Define the kernel between two graphs as the sum of inner products of their label-count vectors over all iterations $i0, 1, \ldots, k$

This process increases the expressive power of the node labels by incorporating progressively larger neighborhoods.
The number of iterations h is a hyperparameter and is typically kept small (e.g., 3–5) to balance expressiveness and efficiency. Iterations stop either when k is reached or when label convergence occurs (i.e., no further changes in node labels).
The WL kernel is considered powerful for several key reasons:

- Expressiveness – Distinguishes many pairs of non-isomorphic graphs that simpler kernels cannot; while not complete for isomorphism testing, it is highly effective in practice.

- Scalability – Computationally efficient, with time complexity linear in the number of edges and iterations, making it suitable for large graphs.
- Structure-awareness – Captures both local and global structural properties by adjusting h which defines the receptive field.
- Versatility – Supports labeled or unlabeled graphs and can be combined with node attributes or edge information through extensions.

4.3 Graph Neural Networks

Graph Neural Networks (GNNs) are a class of neural networks specifically designed to process data represented as graphs, where the relationships between entities are as important as the entities themselves. GNNs are effective for tasks at three distinct levels:

Node-level: Node classification, Node regression
Edge-level: Link prediction, Edge classification, Edge Weight prediction
Graph-level: Graph classification, Graph prediction

To perform graph-level tasks such as seizure classification effectively, a network must be capable of distinguishing between non-isomorphic graphs- which is fundamentally linked to the graph isomorphism test.
Thus, a network with greater discriminative capability generally exhibits superior performance. It becomes imperative to improve GNNs discriminative methods

Message Passing

Message Passing Neural Network (MPNN) is a common type of GNN framework which models computation as an iterative process of exchanging and aggregating information among neighbour nodes i.e nodes which have an edge between them. MPNNs can be viewed as a learnable generalization of certain graph kernels [11].
It is characterised by three key components:

1. **Message function**
   Generates a message from neighbor u to target node v at step i ($i \in 1, ...k$ denotes the message passing iteration or hop)
   $$m^{(i)}_{u \to v} = M_i(h^i_u, h^i_v)$$
   where
   h^i_v = feature vector of target (self) node v at step i
   h^i_u = feature vector of neighbour node u at step i

   Usually, the function is a linear transformation with a non-linear activation
   $$M_i(h^i_u, h^i_v) = \sigma(W_{u,v} \cdot h^i_u + b_m)$$
   for some learnable neighbour weight $W_{u,v}$ and bias b_m

2. **Aggregate Function**
   Combines all the messages from neighbourhood of v; $\forall v \epsilon V(G)$
   $$m^{i+1}_v = AGGREGATE(m^i_{u \to v} | u \epsilon \mathcal{N}(v))$$
   The choice of aggregation may be a summation, mean pooling,max pooling etc.

3. **Update Function**
   Updates the node's state using the aggregated message
$$h_v^{i+1} = U_i(h_v^i, m_{u\to v}^i)$$

At each hop, a node aggregates information from its immediate neighbors, thereby expanding its receptive field by one edge. By stacking k message-passing layers, each node's representation incorporates information from nodes up to k hops away in the graph, enabling it to capture increasingly global structural and contextual patterns. Thus h_v^{k+1} contains feature information of nodes in the k-hop neighbourhood $\mathcal{N}^k(v)$ (set of nodes reachable from v by a path of length at most k).

Message passing is a powerful framework because:

- It enables models to leverage both the attributes and the structure of graphs, capturing the dependencies and patterns that arise from graph connectivity
- Locality and scalability:It scales well to large graphs, as computations are localized and can be parallelized across nodes.
- Expressiveness:With enough layers and expressive aggregation functions, message passing GNNs can approximate functions over graphs and even emulate certain classical graph algorithms.

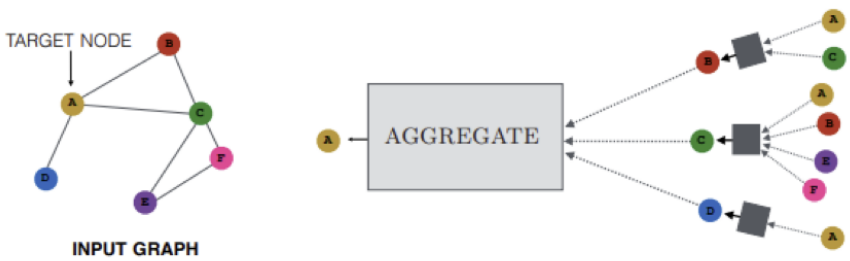


Figure 1: Message Passing Algorithm.

. However, it also has limitations such as oversmoothing (node representations becoming indistinguishable after many layers) and limited expressivity compared to higher-order graph models or graph transformers. The subsequent section outlines how these principles are operationalized in our model design.

5 Literature Review

For a graph-level task such as seizure prediction, improvements in classification accuracy are of paramount importance. GNNs have demonstrated strong capabilities in processing graph-structured data, particularly in domains where relationships between entities carry essential predictive information. In this study, we explored enhancements to GNN performance through three primary avenues:

5.1 Adjacency matrix Construction

EEG signals often lack ground-truth physical connectivity information between electrodes. While a two-dimensional Euclidean distance matrix (physical distance between two electrodes) is frequently used to define adjacency, it may not reflect statistically meaningful relationships between electrodes.

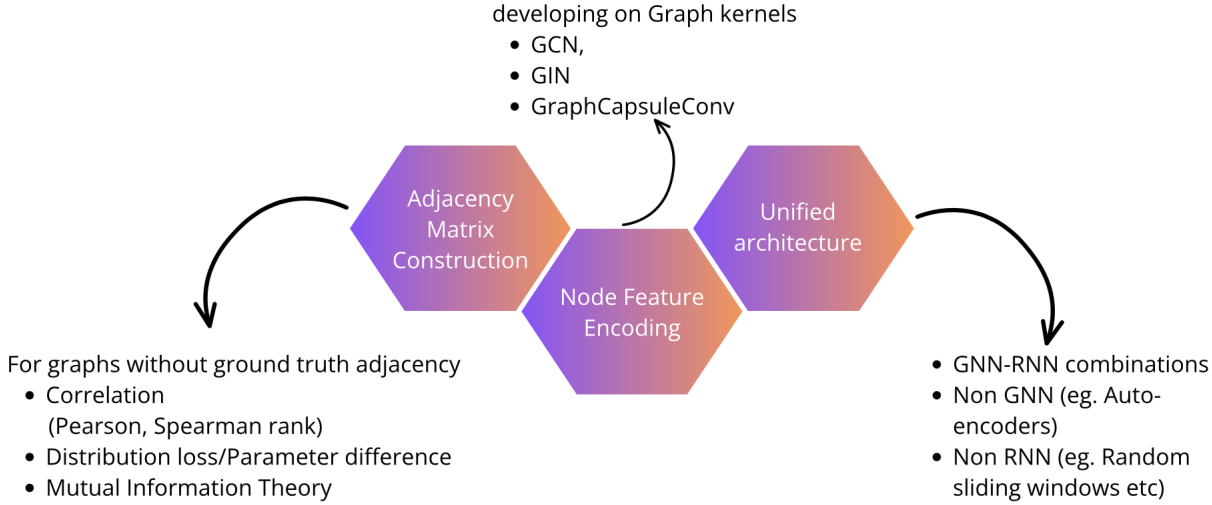


Figure 2: Different Avenues for exploration of GNNs

Statistical connectivity is especially important in EEG analysis, as it helps distinguish genuine neural activity from background noise and understand functional relationships between different brain regions. Several such statistical adjacency measures were investigated in this work, including:

5.1.1 Pearson Correlation

The Pearson correlation coefficient (r r) is a statistical metric that quantifies the strength and direction of the linear relationship between two continuous variables. In the context of EEG analysis, it is a non-parametric computation between the signals recorded at each pair of electrodes to estimate functional connectivity.

$$r = \frac{[(x_i - x)(y_i - \bar{y})]}{[((x_i - x))((y_i - \bar{y}))]}$$

An undirected edge is defined between two electrodes if their correlation coefficient exceeds a predefined threshold $\theta =$ e.g., 0.6 or 0.7), resulting in a weighted or binarized adjacency representation of the EEG network. The method is computationally efficient and highly interpretable; however, it is inherently limited to capturing linear dependencies and may fail to represent complex, non-linear interactions that can arise from the brain's intricate neural dynamics.

5.1.2 Spearman Rank Correlation

Spearman's rank correlation coefficient, denoted as ρ_s (population) or r_s (sample), is a non-parametric statistical measure that quantifies the strength and direction of a monotonic relationship between two variables. first converts the data values into their respective ranks and then computes Pearson's correlation on these ranks. Given two variables X and Y (in the absence of tied ranks) the coefficient can be computed using the formula:

$$r_s = 1\frac{6\sum_{i=1}^{n} d_i^2}{n(n^2-1)}$$

where $d_i = R(X) - R(Y)$ is the difference between the ranks of the ith pair, and n is the number of observations.

5.1.3 Distribution Parameter difference

Assuming that the EEG time series recorded at each electrode can be approximated by a univariate normal distribution, the activity at electrode i is modeled as $X_i \sim N(\mu_i, \sigma_i^2)$ where μ_i and σ_i^2 denote the mean and variance of the signal, respectively. To quantify the similarity (or dissimilarity) between two electrodes i and j one can compare their corresponding distributions $N(\mu_i, \sigma_i^2)$ and $N(\mu_j, \sigma_j^2)$
Euclidean distance is a simple approach that computes a weighted distance between the means and variances:

$$d(i,j) = \sqrt{(\mu_i - \mu_j)^2 + (\sigma_i^2 - \sigma_j^2)}$$

Any distance below a threshold θ implies a statistical edge between the two electrodes. Thus this implies that the distributions of the activity observed in the two electrodes are similar, and generalizes the conditions for edge generation more than other methods.

5.1.4 Mutual Information

Mutual Information (MI) is an information-theoretic measure that quantifies the statistical dependency between two random variables, capturing both linear and non-linear relationships. Given two EEG channels X and Y, their mutual information is defined as:

$$I(X;Y) = \sum_{x \epsilon X} \sum_{y \epsilon Y} p(x,y) \cdot log \frac{p(x,y)}{p(x)p(y)}$$

where $p(x,y)$ denotes the joint probability distribution of (X,Y), and $p(x)$ and $p(Y)$ are the corresponding marginal distributions. A higher value of $I(X;Y)$ indicates a stronger statistical dependency between the two signals.
The adjacency matrix $A \epsilon \mathbb{R}^{n \times n}$ is computed as:

$$a_{i,j} = \begin{cases} I(X_i, Y_j) & \text{if } I(X_i; Y_j) > \theta \\ 0 & \text{otherwise} \end{cases}$$

This approach offers several advantages over correlation-based methods. Unlike Pearson's r which only captures linear associations, mutual information accounts for both linear and non-linear dependencies, making it more suitable for modeling the complex interactions underlying brain activity. However, the method is computationally more expensive, and the choice of probability estimation technique and threshold θ can significantly influence the resulting graph structure.

5.2 Node Feature encoding

Effective utilization of graph data requires encoding node, edge, or graph-level features into suitable numerical representations. These representations serve as the foundation for downstream tasks such as classification, clustering, and link prediction. Following are some methods investigated for node feature encoding.

5.2.1 GCNConv

The Graph Convolutional Network (GCN), based upon the paper by Kipf and Welling (2017)[13], is one of the most widely used message passing architectures for learning on graphs. The GCN layer generalizes the convolution operation from Euclidean domains (e.g., images) to non-Euclidean graph-structured data. the layer updates each node's embedding by taking a weighted average of its own features and those of its neighbors, followed by a learnable linear transformation.

Formally, given a graph $G = (V, E)$ with n nodes, an input feature matrix $X \in \mathbb{R}^{n \times d}$, and an adjacency matrix A, the layer computes node embeddings by aggregating information from each node's neighbors. The propagation rule of a single GCN layer is defined as:

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-\frac{1}{2}}\tilde{A}\tilde{D}^{-\frac{1}{2}}H^{(l)}W^{(l)}\right),$$

where:

- $H^{(l)} \in \mathbb{R}^{n \times d_l}$ is the node representation at layer l (with $H^{(0)} = X$),
- $W^{(l)} \in \mathbb{R}^{d_l \times d_{l+1}}$ is a trainable weight matrix,
- $\tilde{A} = A + I_n$ is the adjacency matrix with added self-loops,
- $\tilde{D}$ is the diagonal degree matrix of $\tilde{A}$,
- $\sigma(\cdot)$ is a nonlinear activation function such as ReLU.

5.2.2 GINConv

The Graph Isomorphism Network (GIN), introduced by Xu et al. (2019)[14], was proposed to address the expressivity limitations of earlier Graph Neural Networks (GNNs). GINs were designed to match the discriminative power of the Weisfeiler–Lehman (WL) graph isomorphism test.

Unlike the Graph Convolutional Network (GCN), which normalizes neighbor contributions, GIN aggregates neighbor features and then applies a Multi-Layer Perceptron (MLP) for transformation. The propagation rule of a single GIN layer is given by:

$$h_v^{(k)} = \mathrm{MLP}^{(k)}\Big((1 + \epsilon^{(k)}) \cdot h_v^{(k-1)} + \sum_{u \in \mathcal{N}(v)} h_u^{(k-1)}\Big),$$

where:

- $h_v^{(k)}$ is the embedding of node v at layer k,
- $h_v^{(0)}$ is the initial feature vector of node v,
- $\mathcal{N}(v)$ denotes the set of neighbors of node v,
- $\epsilon^{(k)}$ is either a learnable parameter or a fixed scalar that balances the contribution of the node's own features relative to its neighbors,
- $\mathrm{MLP}^{(k)}$ is a multi-layer perceptron applied after aggregation.

GIN is the use of *sum aggregation* combined with an expressive function approximator (MLP). This choice ensures that GIN can distinguish graph structures as effectively as the WL test, which many earlier GNNs fail to achieve due to their averaging or normalization operations. However, this increased expressivity may come at the cost of reduced stability in very large or noisy graphs, as the sum aggregation can be sensitive to variations in node degree.

5.2.3 GraphCapsule

The Graph Capsule Network (GCAPS-CNN), proposed by Verma and Zhang (2018)[17] , generalizes the concept of capsule networks to graph-structured data. Capsule networks, originally developed for image analysis, address the limitations of scalar neuron activations by representing features as vectors (or higher-order capsules). These capsules capture not only the presence of a feature but also its instantiation parameters, such as orientation, scale, or spatial relationships.

In the context of this work, the graph capsule function replaces the scalar aggregation of standard graph convolution with a vector-valued representation based on higher-order statistical moments , which are then passed through an MLP to obtain a graph embedding.

The function is defined as:

$$f_i(x_0, x_1, \ldots, x_k) = \frac{1}{|\mathcal{N}(i)|} \begin{bmatrix} \sum\limits_{k \in \mathcal{N}(i)} a_{ik} x_k \\ \sum\limits_{k \in \mathcal{N}(i)} a_{ik} x_k^2 \\ \vdots \\ \sum\limits_{k \in \mathcal{N}(i)} a_{ik} x_k^p \end{bmatrix},$$

where a_{ik} denotes the edge weight between nodes i and k, and p is the number of statistical moments considered.

Consequently, for an input feature matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$ and adjacency $\mathbf{A}$, the graph capsule network produces an output:

$$f(\mathbf{X}, \mathbf{A}) \in \mathbb{R}^{N \times h \times p},$$

where h is the hidden dimension and p represents the capsule dimension corresponding to the order of statistical moments.

The principal contribution of GCapsNets lies in their ability to preserve higher-order structural and feature-level dependencies through capsule representations and adaptive routing. This enables them to model hierarchical substructures in graphs more effectively than conventional message-passing GNN layers.

The referenced paper demonstrates that GCapsNets achieve competitive results in graph classification tasks, particularly when capturing nuanced relationships between graph substructures is critical. While increased feature descriptors are powerful, the statistical moments are not injective, and hence there is no direct comparison to the WL-isomorhpism test.

5.3 Unified Architecture

Models often differ in how they process structural and attribute information. A unified architecture refers to a framework that integrates multiple functional components such as feature extraction, structural encoding, and task-specific prediction—within a single end-to-end design. The categorization strategy adopted in this work follows the taxonomy proposed by Ma et al. [15], which organizes dynamic graph learning methods based on their temporal and structural modeling characteristics.

While graph-based models were categorised into 4 groups. Certain architectures (e.g., Variational Graph Autoencoders, EvolveGCN etc.) were studied at the conceptual level but not experimentally validated in this work. Their inclusion in the review aims to situate the proposed method within the broader research landscape.

5.3.1 GNN-RNN Combinations

The intuitive strategy for processing spatio-temporal data is to employ Graph Neural Networks (GNNs) for modeling spatial dependencies and Recurrent Neural Networks (RNNs) for capturing temporal dynamics in a separate manner. The integration of these two components can be achieved through the following combination strategies.

Time then Graph

Temporal information is first processed (e.g., using a sequence model like RNN or Transformer) to generate time-aware node features, which are then fed into a GNN for spatial (graph) processing [4]. In this approach, the graph's connectivity is presumed to be static, thereby neglecting valuable information inherent in the evolving structural relationships within the graph.
Algorithm:

1. Process Node features X_n^t for n nodes over time t using a sequence model (Eg. RNN) to obtain a temporal embedding for each node

$$h_n^t = \text{RNN}(X_n^{1:t})$$

2. Apply GNN to aggregate messages from each nodes neighbourhood.

$$z_v = \text{GNN}(h_v^t, \{h_u^t, u \in N(v)\})$$

3. Obtain new hidden embedding h^t
4. At the end of time sequence, use the final embedding for classification.

Time and Graph

It processes temporal and graph information simultaneously, often interleaving temporal and spatial operations [4]. This assumes that the graphs at each window are independent, without any mechanism to capture temporal dependencies between graphs.
Algorithm:

1. Divide entire sequence data into windows 1 to t
2. Concatenate current node embeddings X^t at time window t with the hidden embedding h^{t-1} obtained in previous window

$$X_n^t = (X_n^t || h^{t-1})$$

3. Apply GNN to aggregate messages from each nodes neighbourhood.

$$z_v = \text{GNN}(h_v^t, \{h_u^t, u \in N(v)\})$$

4. Obtain a Graph embedding and classify

Graph-then-Time

spatial information is first processed using a GNN to capture graph structure for each window, followed by temporal modeling (e.g., RNN or Transformer) of the graph embeddings obtained [4].However, it lacks a specific mechanism to incorporate the interdependence of graph structures across time steps.
Algorithm:

1. Divide entire sequence data into windows 1 to t
2. Apply GNN to aggregate messages from each nodes neighbourhood for each window.

$$h_v^t = \mathrm{GNN}(x_v^t, \{x_u^t, u \in N(v)\})$$

3. Obtain a Graph embeddings for each window. This is a sequence data
4. Apply sequence model (Eg. RNN) to the graph embeddings. Use the final embedding for classification.

$$z^t = \mathrm{RNN}(h^{1:t})$$

5.3.2 Structural Specific Models

Structural-specific models are a class of dynamic graph learning methods that emphasize the modeling of graph topology, node interactions, and connectivity patterns across time steps — typically using graph-centric mechanisms (e.g., graph autoencoders, attention mechanisms, random walks) without relying heavily on sequence models like RNNs or temporal convolutions.
For instance, the Variational Graph Recurrent Neural Network (VGRNN) [7] models dynamic graphs using a variational autoencoder framework with recurrent updates.

5.3.3 Temporal Specific Models

Temporal-specific models in dynamic graph learning emphasize capturing the evolution of node and edge states across time, complementing GNNs for spatial structure with temporal mechanisms such as convolutions, attention, or memory modules. These approaches focus on maintaining temporal dependencies, improving scalability through efficient updates, and adapting to diverse application domains. Unlike structural-specific models, which highlight topological patterns, temporal-specific methods prioritize the progression of dynamic interactions. A representative example is the Temporal Graph Network (TGN), which employs memory and attention to model long-range temporal dependencies effectively.

6 Methodology

6.0.1 Motivation for specific model choices

The proposed architecture integrates temporal and graph-based components following a *Time-then-Graph* paradigm. This choice was motivated by prior findings in *Dynamic Multi-Channel EEG Graph Modeling for Time-Evolving Brain Network* (anonymous authors, under double-blind review), which demonstrated that among the three possible GNN–RNN integration strategies (Time-then-Graph, Graph-then-Time, and Parallel), the Time-then-Graph approach must achieve the best performance for EEG classification tasks. Building on this insight, our implementation first applies LSTMs to capture temporal dynamics at the channel level, followed by graph neural networks to model spatial dependencies across EEG channels.

We modified the GIN architecture by incorporating five parallel MLPs, whose outputs are concatenated before being passed to the classifier. This design choice was made to ensure that the GIN layer possesses a comparable depth and parameter count to the GraphCapsule model since a simple GIN layer performed similar to a standard GCN, which we attribute to the relatively small graph size and the limited amount of available data.

The objective is to systematically investigate the impact of the different methodological variations discussed earlier, assessing whether they contributed to improvements in overall model performance.

6.0.2 Data Preprocessing

The dataset used in this work originates from the *Seizure Prediction Challenge* (Kaggle), provided by the American Epilepsy Society. It consists of multi-channel EEG recordings from two patients, with two classes: interictal and preictal. Each labeled file is a $15 \times 300{,}000$ matrix, corresponding to 15 channels recorded over 10 minutes. The data split includes 10 files for training, 4 for validation, and 4 for testing, per patient.

To facilitate training, each 10-minute signal was segmented into non-overlapping windows of 2 seconds (corresponding to 10,000 time steps), which were treated as independent input samples. This segmentation both increases the number of training samples and enables the model to focus on localized temporal patterns relevant to seizure prediction.

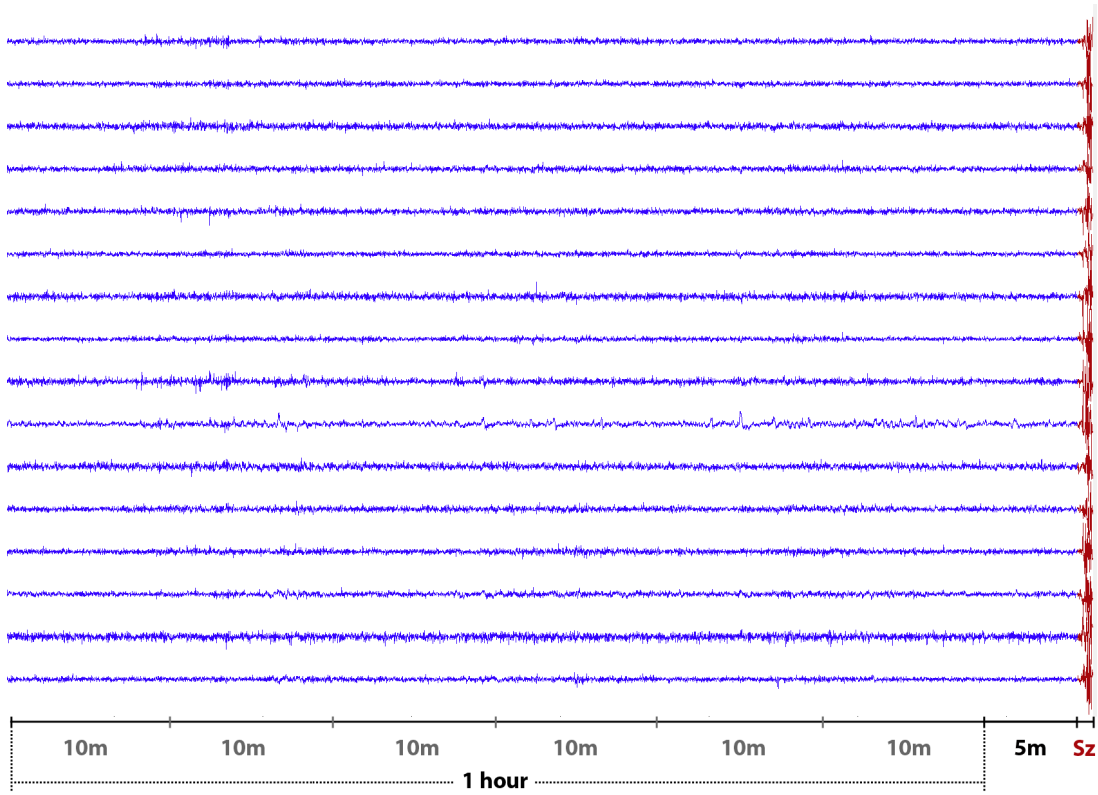


Figure 3: A sample of raw EEG signals where each row represents a channel's signal recorded over 10 minutes at 5000 Hz.

6.0.3 Graph Construction

Spatial dependencies between EEG channels were modeled using a correlation-based graph. Four types of Adjacencies was computed between channel pairs, and an edge was included if the absolute correlation exceeded a threshold $\theta \in [0.3, 0.6]$. Each adjacency matrix was then used as the graph structure for subsequent GNN layers. By leveraging correlation-based connectivity, the graph encodes potential functional relationships between EEG channels.

6.0.4 Model Architecture

The proposed architecture integrates temporal and graph-based components as follows:

1. **Convolutional Neural Network (CNN):** Two and three-layer CNN modules were applied to extract local temporal features from EEG windows. The CNN layers map raw signals into an 7-8-dimensional representation, with nonlinear activation functions (`tanh` and `ReLU`) applied between layers.

2. **Long Short-Term Memory (LSTM):** The extracted features were processed with an LSTM module (hidden dimension = 11-16) to capture long-range temporal dependencies within each window.

3. **Graph Neural Network (GNN):** The outputs of the LSTM were projected onto the EEG channel graph, where a GNN (hidden dimension = 7-12) aggregated information across correlated channels. Multiple variants were evaluated, including Graph Convolutional Networks (GCN), a modified Graph Isomorphism Network (GIN), and a Graph Capsule Network.

4. **Classification Layer:** The final node embeddings were pooled and passed to a fully connected linear layer for binary classification (interictal vs. preictal).

6.0.5 Training Setup

The model was trained with the following configuration:

- Loss function: Cross-Entropy Loss
- Optimizer: Adam
- Learning rate: 1×10^{-3}, decayed to 1×10^{-4}
- Weight decay: 1×10^{-4}
- Training steps: 10,000–25,000 iterations
- Bias layers: None

This pipeline enables the joint modeling of temporal dynamics (via CNN and LSTM) and spatial relationships (via GNN) in EEG signals, providing a flexible framework to study seizure prediction tasks. Figure 16 provides an overview of the proposed architecture.

7 Results

For evaluation, the models obtained at the end of 30 training epochs were considered for each category. The experimental setup is summarized below:

- Training dataset size: **3,930 samples**
- Test dataset size: **2,900 unseen samples**
- Interictal files used: **18**
- Preictal files used: **18**
- Learning rate: **0.001**
- Weight decay: **0.0001**
- Number of patients: **1**
- Platform: **Google Colab**
- Threshold: 0.6
- Number of epochs: 30
- Seed = 0

GCN-Pearson correlation

Best Validation Accuracy = 46.31% | Corresponding Training Accuracy = 93.08%
Test Accuracy = 52.83% | Test Loss = 2.2442

Table 1: Classification Report

Class	**Precision**	**Recall**	**F1-Score**	**Support**
0 (Interictal)	0.5148	0.9848	0.6761	1450
1 (Preictal)	0.8254	0.0717	0.1320	1450
Accuracy			**0.5283**	2900
Macro Avg	0.6701	0.5283	0.4041	2900

GIN-Pearson correlation

Best Validation Accuracy = 39.50% | Corresponding Training Accuracy = 87.76%
Test Accuracy = 53.03% | Test Loss = 1.6655

Table 2: Classification Report

Class	**Precision**	**Recall**	**F1-Score**	**Support**
0 (Interictal)	0.5170	0.9207	0.6622	1450
1 (Preictal)	0.6384	0.1400	0.2296	1450
Accuracy			**0.5303**	2900
Macro Avg	0.5777	0.5303	0.4459	2900

GraphCapsule + Pearson

Best Validation Accuracy = 38.55% | Corresponding Training Accuracy = 87.51%
Test Accuracy = 48.66% | Test Loss = 1.5028

Table 3: Classification Report

Class	Precision	Recall	F1-Score	Support
0	0.4869	0.5000	0.4934	1450
1	0.4862	0.4731	0.4796	1450
Accuracy			**0.4866**	2990
Macro Avg	0.4865	0.4866	0.4865	2990

GCN-Spearman

Best Validation Accuracy = 38.55% | Corresponding Training Accuracy = 94.07%
Test Accuracy = 43.58% | Test Loss = 7.6281
Train Acc = 0.9407, Val Loss = 5.5003, Val Acc = 0.4618 Loss: 7.6281 — Accuracy: 0.4358

Table 4: Classification Report

Class	Precision	Recall	F1-Score	Support
0	0.1444	0.0261	0.0442	1495
1	0.4647	0.8455	0.5998	1495
Accuracy			**0.4358**	2990
Macro Avg	0.3046	0.4358	0.3220	2990

GIN +Spearman

Best Validation Accuracy = 40.46% | Corresponding Training Accuracy = 88.52%
Test Accuracy = 53.14% | Test Loss = 2.4325

Table 5: Classification Report

Class	**Precision**	**Recall**	**F1-Score**	**Support**
0	0.5171	0.9510	0.6699	1450
1	0.6953	0.1117	0.1925	1450
Accuracy			**0.5314**	2990
Macro Avg	0.6062	0.5314	0.4312	2900

GraphCapsule + Spearman

Best Validation Accuracy = 38.30% | Corresponding Training Accuracy = 89.16%
Test Accuracy = 52.07% | Test Loss = 1.9501

Table 6: Classification Report

Class	Precision	Recall	F1-Score	Support
0	0.5114	0.9269	0.6591	1450
1	0.6103	0.1145	0.1928	1450
Accuracy			**0.5207**	2990
Macro Avg	0.5609	0.5207	0.4260	2900

GCN + Distribution Distance

Best Validation Accuracy = 40.71% | Corresponding Training Accuracy = 88.586%
Test Accuracy = 13.71% | Test Loss = 4.0800

Table 7: Classification Report

Class	Precision	Recall	F1-Score	Support
0	0.0073	0.0054	0.0062	1495
1	0.2128	0.2689	0.2376	1495
Accuracy			**0.1371**	2990
Macro Avg	0.1100	0.1371	0.1219	2990

GIN + Distribution Distance

Best Validation Accuracy = 41.86% | Corresponding Training Accuracy = 88.55%
Test Accuracy = 17.21% | Test Loss = 5.2405

Table 8: Classification Report

Class	Precision	Recall	F1-Score	Support
0	0.0052	0.0034	0.0041	1450
1	0.2548	0.3407	0.2915	1450
Accuracy			**0.1721**	2900
Macro avg	0.0948	0.1128	0.1028	2900 height

GraphCapsule + Distribution Distance

Best Validation Accuracy = 36.96% | Corresponding Training Accuracy = 91.98%
Test Accuracy = 11.28% | Test Loss = 6.3103

Table 9: Classification Report

Class	Precision	Recall	F1-Score	Support
0	0.0096	0.0076	0.0085	1450
1	0.1801	0.2179	0.1972	1450
Accuracy			**0.1128**	2900
Macro Avg	0.0948	0.1128	0.1028	2900

GCN + Mutual Information

Best Validation Accuracy = 41.86% | Corresponding Training Accuracy = 89.26%
Test Accuracy = 16.02% | Test Loss = 4.5394

Class	Precision	Recall	F1-Score	Support
0	0.0076	0.0055	0.0064	1450
1	0.2205	0.2814	0.2473	1450
Accuracy			**0.1434**	2900
Macro Avg	0.1141	0.1434	0.1268	2900

GIN + Mutual Information

Best Validation Accuracy = 40.90% | Corresponding Training Accuracy = 87.76%
Test Accuracy = 14.34% | Test Loss = 3.8801

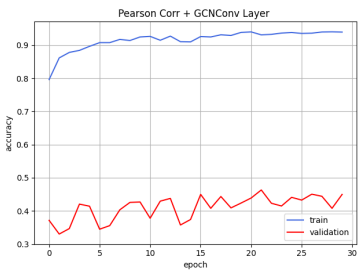
Class	Precision	Recall	F1-Score	Support
0	0.0076	0.0055	0.0064	1450
1	0.2205	0.2814	0.2473	1450
Accuracy			**0.1434**	2900
Macro Avg	0.1141	0.1434	0.1268	2900

GrapCapsule + Mutual Information

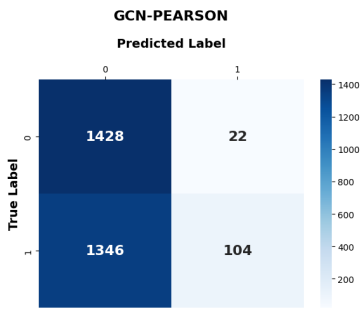
Best Validation Accuracy = 39.25% | Corresponding Training Accuracy = 89.52%
Test Accuracy = 54.69% | Test Loss = 2.1550

Table 10: Classification Report

Class	Precision	Recall	F1-Score	Support
0	0.5277	**0.8938**	**0.6636**	1450
1	**0.6532**	0.2000	0.3062	1450
Accuracy		**0.5469**		**2900**
Macro Avg	0.5904	0.5469	0.4849	2900

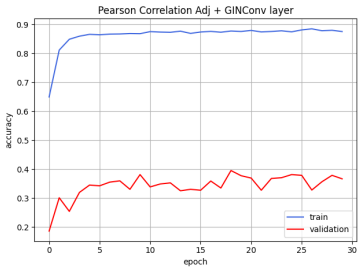


(a) Model Training and Validation

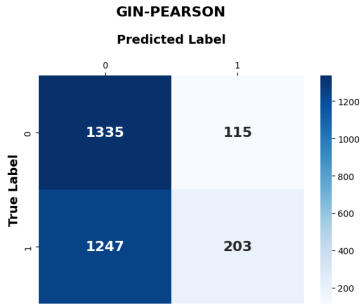


(b) Confusion matrix

Figure 4: GCN Layer combined with Pearson Correlation Adjacency

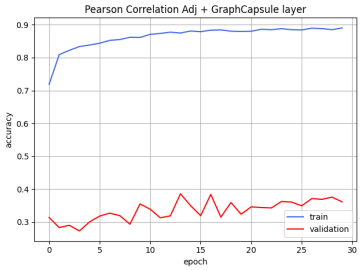


(a) Model Training and Validation

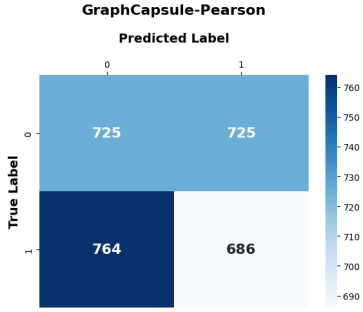


(b) Confusion matrix

Figure 5: GIN Layer combined with Pearson Correlation Adjacency

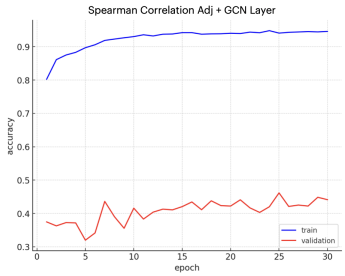


(a) Model Training and Validation

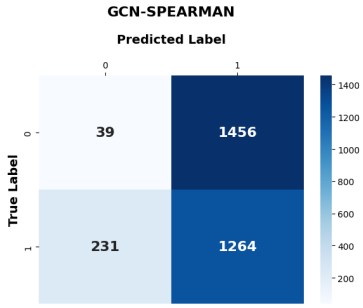


(b) Confusion matrix

Figure 6: GraphCapsule Layer combined with Pearson Correlation Adjacency

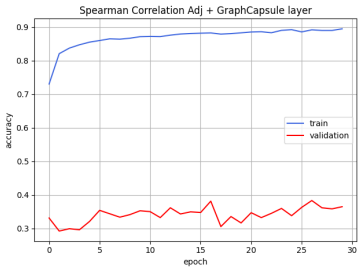


(a) Model Training and Validation

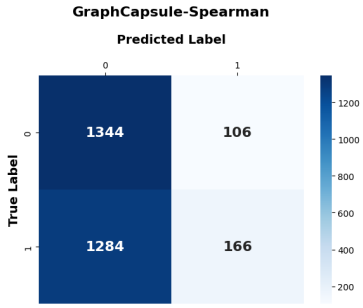


(b) Confusion matrix

Figure 7: GCNConv Layer combined with Spearman Correlation Adjacency

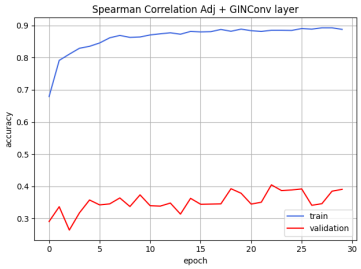


(a) Model Training and Validation

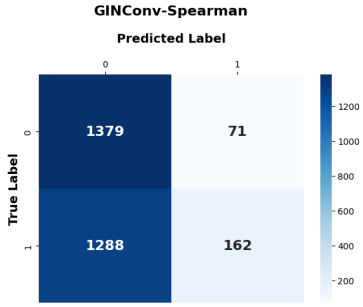


(b) Confusion matrix

Figure 9: GraphCapsule Layer combined with Spearman Correlation Adjacency

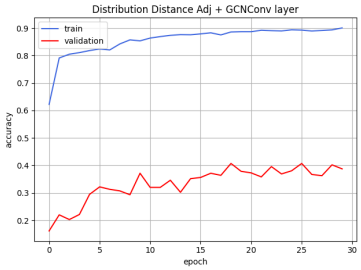


(a) Model Training and Validation

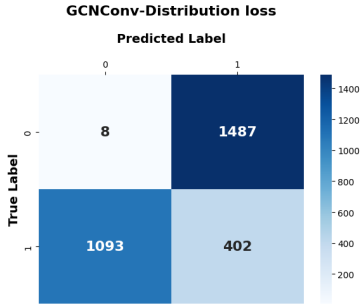


(b) Confusion matrix

Figure 8: GINConv Layer combined with Spearman Correlation Adjacency

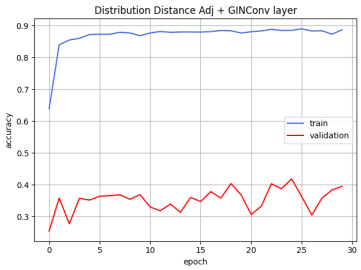


(a) Model Training and Validation

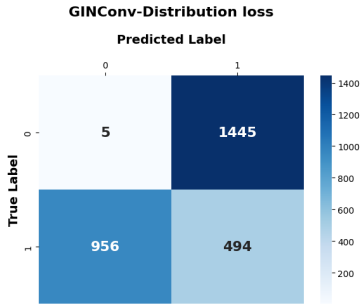


(b) Confusion matrix

Figure 10: GCNConv Layer combined with Distribution Distance Adjacency

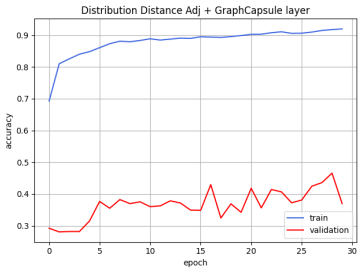


(a) Model Training and Validation

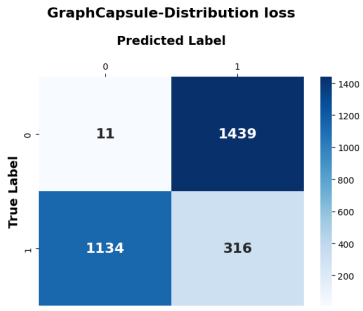


(b) Confusion matrix

Figure 11: GINConv Layer combined with Distribution Distance Adjacency

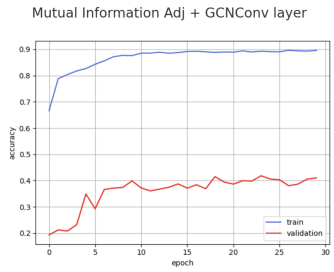


(a) Model Training and Validation

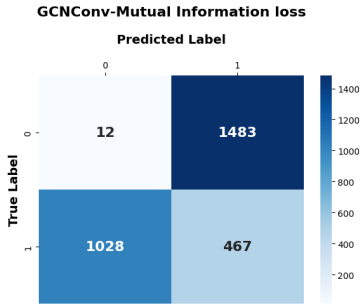


(b) Confusion matrix

Figure 12: GraphCapsule Layer combined with Distribution Distance Adjacency

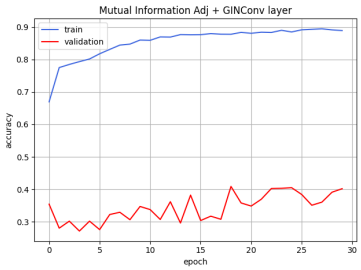


(a) Model Training and Validation

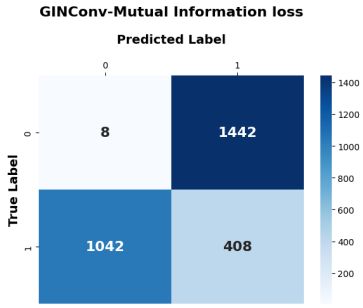


(b) Confusion matrix

Figure 13: GCNConv Layer combined with Mutual Information Adjacency

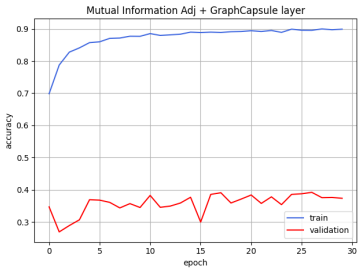


(a) Model Training and Validation

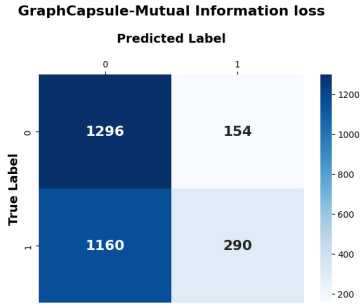


(b) Confusion matrix

Figure 14: GINConv Layer combined with Mutual Information Adjacency



(a) Model Training and Validation



(b) Confusion matrix

Figure 15: GraphCapsule Layer combined with Mutual Information Adjacency

8 Conclusion

The comparative evaluation of GCN, GIN, and GraphCapsule across four adjacency construction strategies (Pearson correlation, Spearman correlation, distribution distance, and mutual information) provides insights into both model behavior and the limitations of the current experimental setup.

When examining architectural differences, both GCN and GIN display relatively consistent performance under correlation-based adjacency matrices, with test accuracies around 52–53%. GIN marginally outperforms GCN, likely due to its increased representational capacity. GraphCapsule, despite its theoretical ability to capture higher-order relationships, does not exhibit any consistent advantage in this context; in fact, its performance under Pearson correlation falls below that of the simpler GCN. These results suggest that, at least within the constraints of the current dataset and adjacency choices, GraphCapsule's added complexity does not yield measurable benefits.

When comparing adjacency strategies across models, Pearson and Spearman correlation yield the most reliable results, with test accuracies near 53%. These methods appear to preserve some linear dependencies between EEG channels that provide a weak discriminatory signal. In stark contrast, the distribution distance adjacency consistently collapses performance across all models, producing accuracies in the 11–17% range—well below chance level. This strongly indicates that distributional similarity is not an appropriate measure for capturing inter-channel relationships in EEG time series. Mutual information–based adjacency performs somewhat better than distribution distance, reaching accuracies of 14–16%, but still remains far below the baseline provided by correlation-based methods. The results suggest that while

mutual information can, in principle, capture non-linear dependencies, in this experimental setup it does not translate into usable graph structure, possibly due to noise, high variance in MI estimates, or the limited amount of data per channel pair.
However, beyond these relative comparisons, it is important to emphasize that the absolute performance of all models is abnormally low. Even the best-performing configurations barely exceed 53% accuracy, only marginally above random guessing in a balanced binary classification problem (50%). In some cases, such as with distribution distance or mutual information graphs, performance is worse than random. This outcome calls into question the practical significance of architectural or adjacency-based variations: small differences between GCN, GIN, and GraphCapsule are largely overshadowed by the overall failure of the models to generalize meaningfully.
A possibility of Overparametrising the models may explain the overall low accuracies. With relatively deep or complex architectures, the models may have contained far more parameters than could be supported by the limited dataset, leading to poor generalization. This is reflected in the observed tendency of several configurations to bias toward predicting a single class, despite the fact that the dataset was balanced with an equal number of samples per class. Such behavior suggests that the optimization process exploited spurious shortcuts rather than meaningful inter-channel patterns, further underscoring the mismatch between model capacity and available data.

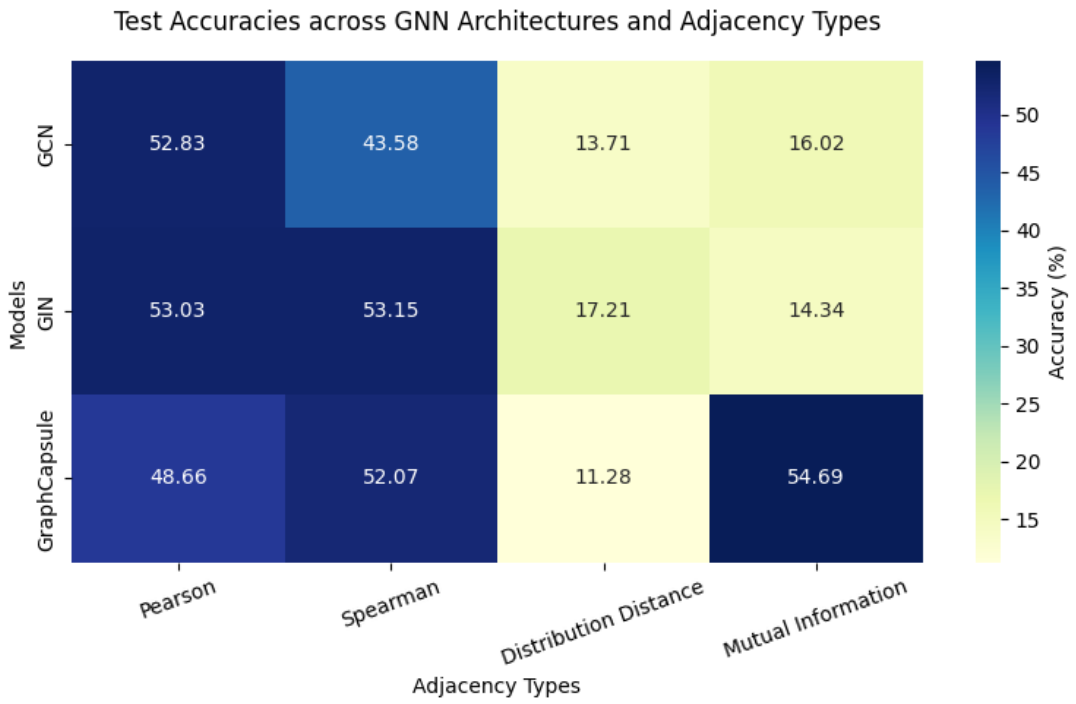


Figure 16: Heatmap of accuracies across all strategies investigated.

9 Future Scope

The current results highlight that the proposed GNN-based models, when applied directly to raw or weakly pre-processed EEG signals, fail to achieve meaningful predictive performance. To address these

limitations, several future directions can be pursued:

1. **Improved Pre-processing and Feature Engineering:** Instead of feeding raw EEG or CNN-random-weight embeddings, clinically relevant features (e.g., bandpower, entropy, Hjorth parameters, Riemannian features) or time–frequency representations can be extracted to ensure physiologically meaningful inputs.

2. **Task-Aligned Architectures:** Lightweight models or temporal architectures (e.g., temporal graph networks, ST-GCNs, graph transformers) that explicitly capture both spatial and temporal dependencies in EEG may improve performance over static GNNs.

3. **Regularization and Hyperparameter Optimization:** Automated hyperparameter tuning, architecture search, and improved weight regularization may prevent overfitting while ensuring gradient signals remain strong.

4. **Alternative Adjacency Constructions:** More robust graph construction techniques, such as adaptive adjacency learning, attention-based edge weighting, or subject-specific dynamic graphs, could better capture the underlying neural connectivity patterns.

5. **Dataset Expansion and Cross-Subject Evaluation:** Training on larger, multi-patient datasets with better class balance would reduce overfitting to patient-specific patterns and allow the model to generalize.

6. **Hybrid Methods:** Combining GNNs with domain-specific priors (e.g., physics-guided neural networks, neural ODEs, or hybrid CNN–GNN approaches) may better model seizure dynamics.

7. **Bias–Variance Tradeoff Analysis:** Systematically studying the bias–variance characteristics of different models may provide insights into whether poor performance stems from underfitting or overfitting. Leveraging controlled increases in bias (e.g., simpler architectures, stronger priors) could in some cases improve generalization compared to overly complex models.

Overall, addressing these issues requires both methodological improvements in model design and principled integration of neuroscientific knowledge into the machine learning pipeline.

References

[1] Fisher, R. S., et al. (2014). ILAE official report: a practical clinical definition of epilepsy. *Epilepsia*, 55(4), 475–482.

[2] Litt, B., & Echauz, J. (2002). Prediction of epileptic seizures. *The Lancet Neurology*, 1(1), 22–30.

[3] Wang, X., et al. (2021). Graph Neural Networks for EEG-based seizure detection. *Journal of Neural Engineering*, 18(4), 026009.

[4] Zheng, Y., et al. (2024). A Survey of Dynamic Graph Neural Networks. *Frontiers of Computer Science*.

[5] American Epilepsy Society Seizure Prediction Challenge. Available at: https://www.kaggle.com/competitions/seizure-prediction

[6] World Health Organization. (2024). Epilepsy. Available at: https://www.who.int/news-room/fact-sheets/detail/epilepsy

[7] Hajiramezanali, E., Hasanzadeh, A., Duffield, N., & others. (2019). Variational Graph Recurrent Neural Networks. *AAAI*.

[8] Vishwanathan, S. V. N., Schraudolph, N. N., Kondor, R., & Borgwardt, K. M. (2010). Efficient graphlet kernels for large graph comparison. *Journal of Machine Learning Research*, 11, 315–346.

[9] Li, C., Fang, G., Liu, Y., Liu, J., & Song, L. (2020). Decoding Silent Reading EEG Signals Using Adaptive Feature Graph Convolutional Network. *IEEE Transactions on Neural Systems and Rehabilitation Engineering*, 28(12), 2810–2820.

[10] Shervashidze, N., Schweitzer, P., van Leeuwen, E. J., Mehlhorn, K., & Borgwardt, K. M. (2011). Weisfeiler-Lehman Graph Kernels. *Journal of Machine Learning Research*, 12, 2539–2561.

[11] Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., & Dahl, G. E. (2017). Neural Message Passing for Quantum Chemistry. In *Proceedings of the International Conference on Machine Learning (ICML)*.

[12] Anastasiadou, M. N., Christodoulakis, M., Papathanasiou, E. S., & Papacostas, S. S. (2019). Graph Theoretical Characteristics of EEG-Based Functional Brain Networks in Patients With Epilepsy: The Effect of Reference Choice and Volume Conduction. *Brain Topography*, 32(4), 603–618.

[13] Kipf, T. N., & Welling, M. (2017). Semi-Supervised Classification with Graph Convolutional Networks. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

[14] Xu, K., Hu, W., Leskovec, J., & Jegelka, S. (2019). How Powerful Are Graph Neural Networks? In *Proceedings of the International Conference on Learning Representations (ICLR)*.

[15] Ma, Y., Wang, S., Tang, J., et al. (2021). A Comprehensive Survey of Dynamic Graph Neural Networks: Models, Frameworks, Benchmarks, Experiments and Challenges. *arXiv preprint arXiv:2106.07154*.

[16] Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., & Dahl, G. E. (2017). Neural Message Passing for Quantum Chemistry. *ICML*. [Duplicate retained for completeness]

[17] Verma, S., & Zhang, Z.-L. (2018). Graph Capsule Convolutional Neural Networks. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

[18] Tang, S., & Dunnmon, J. (2021). Self-Supervised Graph Neural Networks for Improved Electroencephalogram Representation Learning. *arXiv preprint arXiv:2106.01744*.

[19] Gasteiger, J., Weißenberger, S., & Günnemann, S. (2021). Diffusion Improves Graph Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[20] Demir, A., Koike-Akino, T., & Wang, Y. (2021). EEG-GNN: Graph Neural Networks for Classification of Electroencephalogram Signals. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*.

[21] Yang, Z., Cohen, W. W., & Salakhutdinov, R. (2016). Revisiting Semi-Supervised Learning with Graph Embeddings. In *Proceedings of the International Conference on Machine Learning (ICML)*.